Scrape Board

AI-Powered Web Content Analyzer & Management Tool

React · TypeScript · Supabase · Google Gemini · Firecrawl · Deployed to GoDaddy Shared Hosting

http://webscraper.ensconce.in/

Problem / Context

Manual task entry

Lightweight trackers require every task to be typed by hand — slow and repetitive.

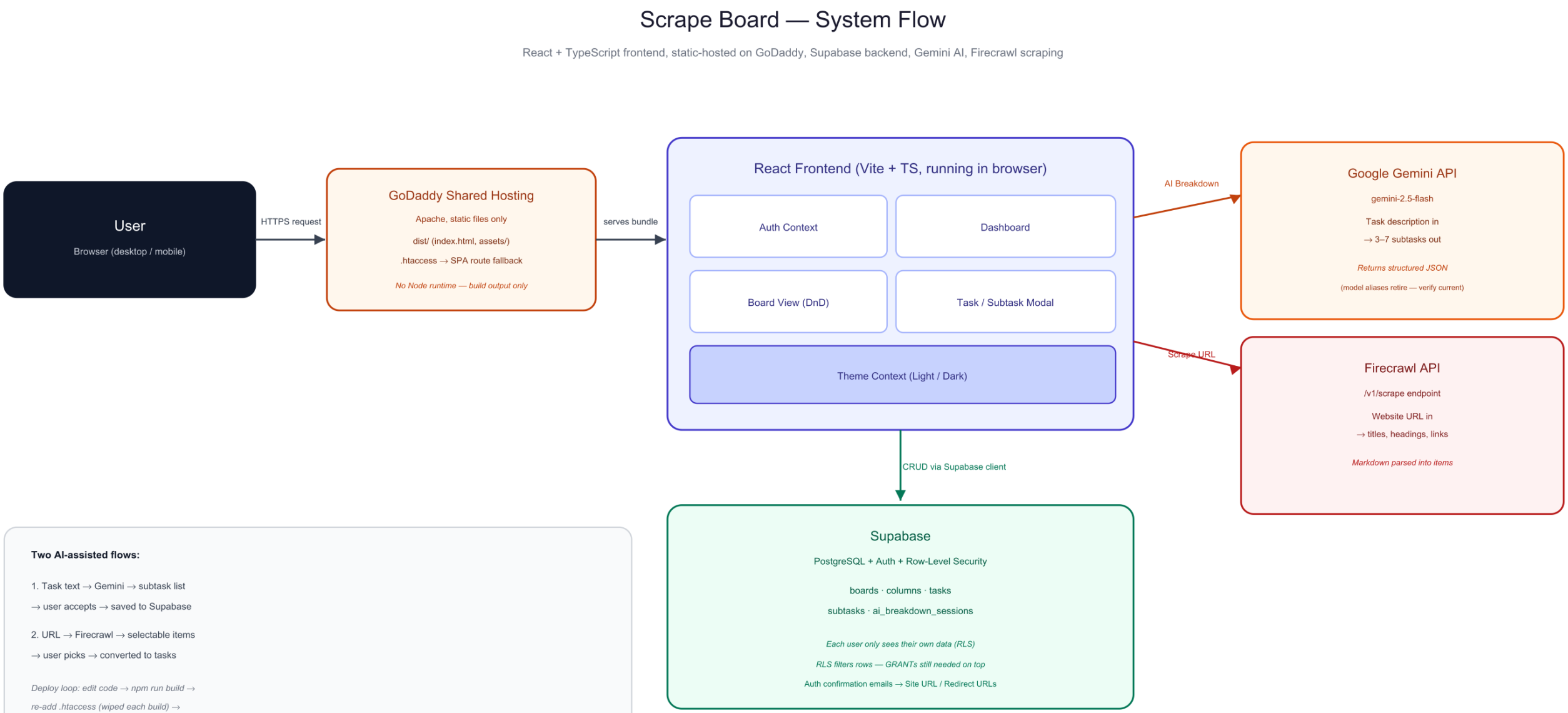
Reference material goes to waste

Turning a spec page, article, or competitor site into to-dos is manual, repeatable work.

Two AI capabilities, one workflow

Test whether generative breakdown and structured web extraction can live inside the core task-manager flow, not as bolt-on tools.

System Flow



Two AI-assisted flows:

1. Task text → Gemini → subtask list
→ user accepts → saved to Supabase

2. URL → Firecrawl → selectable items
→ user picks → converted to tasks

Deploy loop: edit code → npm run build →
re-add .htaccess (wiped each build) →
replace dist/ contents on GoDaddy

Key Decisions & Trade-offs

1

Schema-first, RLS everywhere

Five tables designed before UI; Row-Level Security enabled on every table so authorization lives in Postgres, not the frontend.

2

One contract for both AI/automation paths

Gemini and Firecrawl calls share the same pattern: structured input in, typed output out, fail loudly on bad keys or malformed responses.

3

Client-side API keys — accepted trade-off

Keys are baked into the public bundle for fast, simple shipping. A production version would proxy both through a backend/edge function.

4

Shared hosting instead of Vercel/Netlify

Deployed to existing GoDaddy hosting by treating the Vite build output as what it is — plain static files, no Node runtime required.

5

RLS ≠ GRANTs

Disabling Supabase's table auto-exposure (the secure default) blocks all API access until table privileges are explicitly granted — RLS only filters after access is already permitted.

Getting It Running: Accounts & Deployment

Third-Party Accounts

- Supabase — new project; Enable Data API on, auto-expose tables off; run schema migration; GRANT table privileges to authenticated role
- Firecrawl (firecrawl.dev) — free-tier API key for the /v1/scrape endpoint
- Google Gemini (aistudio.google.com) — free API key; targets gemini-2.5-flash (model aliases retire periodically)
- All keys set as VITE_* vars in .env, baked into the build at compile time

Deploying to GoDaddy Shared Hosting

- npm run build → produces plain static files in dist/ (no Node runtime needed on the host)
- Create a subdomain with its own isolated document root (never share it with the main site)
- Upload only the contents of dist/ — not src/, node_modules/, or .env
- Add .htaccess to fall back to index.html so React Router deep links don't 404 on Apache
- Every rebuild wipes dist/ (including .htaccess) — re-add and re-upload after each change

Outcome

What Held Up

- RLS enforced multi-user isolation even under direct console query attempts
- Defensive parsing on AI responses mattered more than prompt engineering — LLM output shape was the real reliability risk
- Static-file deployment model made shared hosting a viable, low-cost target for a Vite/React SPA

What I'd Do Next

- Move Gemini and Firecrawl keys behind a backend/edge function instead of the public bundle
- Add retry/backoff on AI calls and cache scrape results per URL
- Automate the GoDaddy upload step (currently manual File Manager) with an FTP/SFTP deploy script